Tapwave, Inc. Proprietary

Tapwave®

Tapwave Developer's Overview

Update: The Tapwave product is referred to by the code name "Road Dawg" in this document.

Update: The links do not work correctly in this document.

Current Version	Date	Author
REV 0.6	5/14/2003	RR

Revision History

Version	Date	Description	Author
REV. 0.1	02/21/2003	FIRST VERSION.	MB
REV. 0.2	02/24/2003	INCORPORATE FEEDBACK FROM DC, NUMBER & FORMAT.	MB
REV 0.3B	02/26/2003	DELETE COPYRIGHT INFO, SUPPLEMENT BUILD INSTRUCTIONS	MB
REV 0.4	02/27/2003	INCORPORATE FEEDBACK FROM DC	MB
REV 0.4A	02/28/2003	PROOFREAD, FORMAT, INSERT HYPERLINKS, COPYRIGHT, & NAME CHANGES	MB/MS
REV 0/4B	02/28/2003	FIXED TRADEMARKS & GRAPHIC LINK	MB/MS
REV 0,5	4/30/2003	ADDED REFERENCES TO TWGFX AND PLAYLIST DOCUMENTS	DC
REV 0,6	5/14/2003	UPDATED REFERENCE TO SDK ENVIRONMENT	RR

© Copyright 2003 Tapwave, Inc. All Rights Reserved. Tapwave is a registered trademark of Tapwave, Inc. Palm OS, the Palm logo, Graffiti, HotSync, and PalmSource are registered trademarks of Palm, Inc. Palm, Palm Powered, and the Palm Powered logo are trademarks of Palm, Inc. X-Forge is a trademark of Fathammer, Ltd. All other brands are trademarks or registered trademarks of their respective owners.

Table of Contents

Tapwave Developer's Overview i

Revision History i

1. Overview 1

2. Forward Binary Compatibility 2

3. Fathammer X-Forge 3D Game Engine 3

4. Fathammer X-Forge Core 3

5. Graphics 4

5.1. 2D Graphics 4

5.1.1. X-Forge Core 2D API 4

5.1.2. Palm OS Window Manager 5

5.1.3. Direct Graphics API 5

5.2. 3D Graphics 5

6. Sound 5

6.1. X-Forge Core Audio APIs 6

6.2. Direct Sound APIs 6

6.3. Palm OS Sound APIs 6

7. Networking 7

7.1. X-Forge Game Engine 7

7.2. X-Forge Core 7

7.3. Palm OS 8

8. Input 9

8.1. X-Forge Core 9

8.2. Palm OS User Input 9

8.3. Tapwave Platform Navigator 9

9. Gaming Support 10

10. Tapwave Platform API Cross-Reference 10

11. Development Environment 11

11.1. Tools 11

11.2. Directory Structure 11

11.3. Build Instructions 12

11.3.1. Introduction 12

11.3.2. Build Steps 12

12. Recommendations 14

13. Sample Application 14

1. Overview

Welcome to the Tapwave platform!

Tapwave provides a next-generation gaming platform that combines the best features of the Palm OS® platform with advanced gaming features, including:

- All-inclusive gaming environment with the Fathammer X-Forge™ 3D Game Engine
- 2D and 3D graphics with the Fathammer X-Forge™ Core Libraries
- High-speed, direct-access graphics API
- Digitized and synthesized sound
- Built-in collaborative networking
- Advanced input event queue model
- Large 480x320 pixel screen, with portrait or landscape mode display

The Tapwave platform is a multi-layered system. This document provides an overview of some of the important APIs provided by these layers. You must choose the most appropriate set of APIs for your application, taking into account the following tradeoffs:

- Execution speed vs. development complexity
- Increased functionality vs. code/storage size
- Portability of application code to other platforms

The following figure shows the **Tapwave Road Dawg** system architecture.

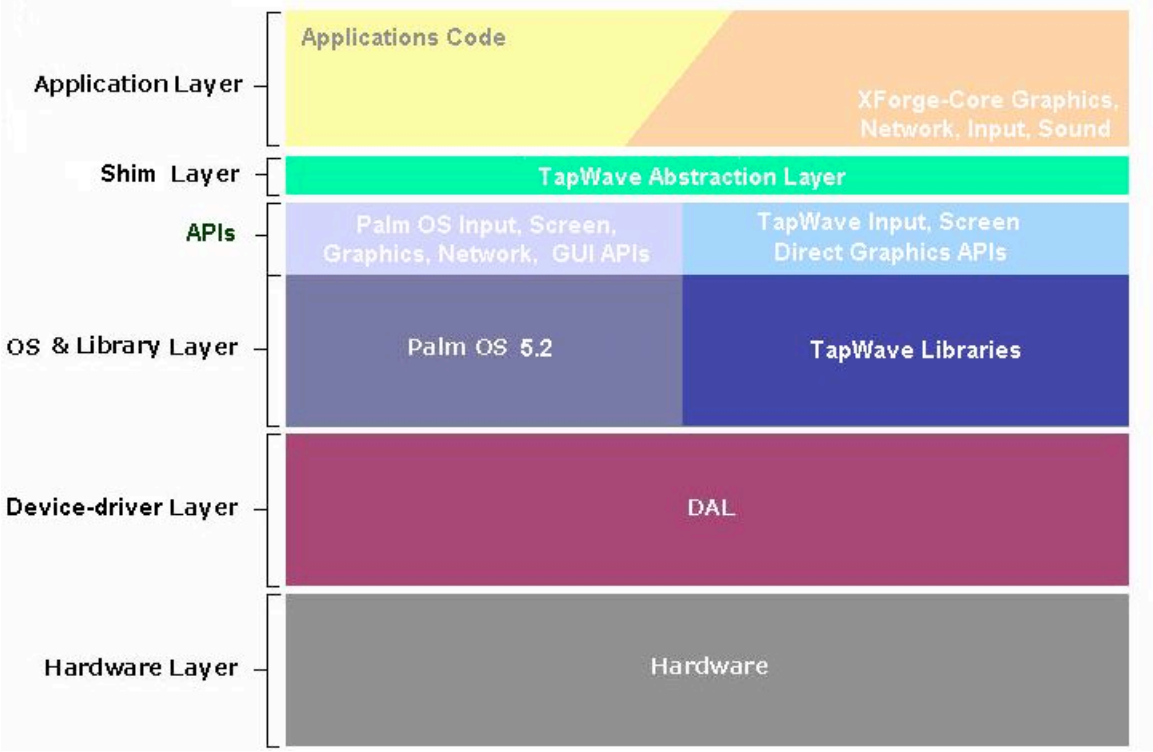


Figure 1. Tapwave Road Dawg System Architecture

2. Forward Binary Compatibility

Tapwave has developed several custom APIs to take advantage of Road Dawg's unique hardware and features. These APIs are written with forward binary compatibility in mind, so that if you write to them you will not have to recode for subsequent versions of the Tapwave platform. If you find that the same functionality is provided by a Tapwave API function and a Palm OS function, for forward compatibility you should call the Tapwave function. For additional information, see "Tapwave Abstraction Layer - Forward Binary Compatibility for Native-ARM Apps" in the *Tapwave Programmer's Reference*.

3. Fathammer X-Forge 3D Game Engine

The Fathammer X-Forge 3D Game Engine provides an all-inclusive environment that supplies most necessary application features. It is a portable framework designed to make 3D game development easier. Among other things, it provides:

- Physics simulations, including gravity and collisions
- Camera angles
- Sound
- Collaborative networking

If your application fits the X-Forge Game Engine model, you should use it. It provides a structure for your application, and it eases development by providing various built-in managers to govern physics, collisions, communications, visibility, and other aspects of your game. Of course, using the X-Forge Game Engine will increase your application's memory and storage usage.

For an overview, see "Chapter 11, Game Engine Overview" in the *X-Forge 1.0.2 Guide*.

The *X-Forge 1.0.2 Guide* for the X-Forge Game Engine, provides extensive and specific information on working with the X-Forge Game Engine, including chapters on 2D and 3D graphics, audio, and networking. Several examples are also included.

4. Fathammer X-Forge Core

The X-Forge Core APIs provide the underlying infrastructure for the X-Forge Game Engine, including:

- Graphics
- Audio
- File I/O
- Memory management
- Resource management
- Networking

While not providing an all-inclusive game engine, the X-Forge Core does provide an application framework, that you can use to structure an application. You should use this framework if it fits your application. If not, you can use the Palm OS model, along with additional features provided by the X-Forge Core libraries.

5. Graphics

The Tapwave platform includes several graphics APIs. These APIs move progressively from the most portable (with the most rapid development), to the least portable (with the most time-consuming development).

The X-Forge Game Engine provides a general framework for game development that frees you to focus on your algorithms and the differentiating factors that make your game unique. Its graphics code is designed with portability in mind, and it provides good graphics performance for most applications.

For some extremely demanding applications, the X-Forge Game Engine may not provide the best possible graphics performance. If you decide not to utilize the X-Forge Game Engine, you will almost certainly want to leverage the extensive functionality provided by the X-Forge Core libraries. The X-Forge Core APIs provide more direct control over display graphics, and offer very good graphics performance.

The Tapwave platform also allows direct access to the hardware graphics accelerator, for those few applications that are extremely speed-sensitive. This is provided by the Direct Graphics API. Because of limited portability, and possible future incompatibility, **Tapwave recommends that you do not use this API**.

Graphics State Note: Interspersing calls to X-Forge Core graphics APIs with calls to other graphics API's is not recommended. The X-Forge Core graphics routines may save some state internally. This could result in a misdrawn or garbled display.

5.1. 2D Graphics

2D graphics support is provided by the following APIs:

- X-Forge Core 2D API
- Palm OS Window Manager
- Direct Graphics API

5.1.1. X-Forge Core 2D API

The X-Forge Core 2D graphics API provides a portable, thin wrapper around the raw pixel data. It supports:

- Blitting with scaling
- Clipping
- Blending
- Rectangle fill
- Line drawing

For more information, see "Chapter 8, 2D Graphics" in the *X-Forge 1.0.2 Guide.*

5.1.2. Palm OS Window Manager

For simpler graphics applications, such as a board game, the Palm OS APIs provide a window management system, including clipping and point transformation.

For more information, see "Drawing on the Palm Powered Handheld" in the *Palm OS Programmer's Companion – Volume I,* at:
http://www.palmos.com/dev/support/docs/palmos/CompanionTOC.html.

5.1.3. Direct Graphics API

The Direct Graphics API provides the maximum graphics performance. Any graphics operation is possible. It provides a frame buffer model—you must know where on the screen you are writing.

The tradeoff for this increased performance is more complex and time-consuming coding, as well as less portable code. For example, since calls to Direct Graphics API do not operate within the Palm OS Window Manager, no window management or clipping is provided—you must code this yourself. Graphics transformations must be hand-coded.

You should utilize the Direct Graphics API only if you have determined that the X-Forge Core and Palm OS graphics routines do not provide the necessary performance for your application, or if your application cannot fit easily into the X-Forge Core graphics model.

For more information, see the "*Tapwave TwGfx Graphics API Reference"* document.

5.2. 3D Graphics

3D graphics on the Tapwave platform are provided through the X-Forge Core graphics library.

For more information, see "Chapter 9, 3D Graphics" in the *X-Forge 1.0.2 Guide*.

6. Sound

As is the case with graphics, you have several sound APIs available, including:

- X-Forge Core Audio API, for sound effects and digitized audio
- Direct Sound API, for synthesized audio
- Palm OS Sound API, for simple sounds and digitized audio

6.1. X-Forge Core Audio APIs

The X-Forge Core Audio APIs were written to make firing one-off sound effects easy, while at the same time supporting more advanced, dynamic control of playback. These APIs provide support for digitized sound, special effects such as Doppler shift, and playback of RIFF Wave files (.wav) and extended Module (.XM) audio sources.

Note: Because digitized sound is more realistic than synthesized sound, it consumes more memory and storage space, and can take longer to play out.

For a platform-independent overview, see "Chapter 3, Platform Independent - Audio" in the *X-Forge 1.0.2 Guide*.

For an overview of the audio capabilities of the X-Forge Core libraries, see "Chapter 5, Overview - Audio" in the *X-Forge 1.0.2 Guide*.

For specific source code examples, see "Chapter 10, Audio" in the *X-Forge 1.0.2 Guide*.

6.2. Direct Sound APIs

The Direct Sound subsystem provides synthesized sound capabilities.

See the "*Direct Sound API"* which will be available at a future date.

6.3. Palm OS Sound APIs

Palm OS Sound APIs support simple sounds such as system beeps and alarms. If you are already using these simple sound routines, Palm OS also supports MIDI digitized sound through ADPCM-sampling.

For more information, see "Chapter 10, Palm System Support - Sound" in the *Palm OS Programmer's Companion – Volume I*, at: http://www.palmos.com/dev/support/docs/palmos/CompanionTOC.html.

For additional overview information, see "Advanced Sound Support" in the *Tapwave Programmer's Reference*.

For API descriptions, see "Advanced Sound API" in the *Tapwave Programmer's Reference*.

7. Networking

Networking features are provided by the:

- X-Forge Game Engine
- X-Forge Core libraries
- Palm OS

Easy collaborative networking is built into the Tapwave platform, simplifying development of multiplayer games. Transport-independence means you only need to code to a single API to enable shared gaming across the room via Bluetooth, or across the Internet via TCP/IP!

For additional overview information specific to the Tapwave platform, see "Bluetooth Collaborative Networking Support" in the *Tapwave Programmer's Reference*.

For API descriptions, see "Part IV, Libraries" in the *Palm OS Programmer's API Reference* http://www.palmos.com/dev/support/docs/palmos/ReferenceTOC.html.

7.1. X-Forge Game Engine

The X-Forge Game Engine provides a robust, high-level network gaming model. Individual game objects on a device can directly interact with game objects on other devices. It provides a multiplayer game model, where the game state between the various devices participating in the game is automatically synchronized, and also includes a method for authenticating and keeping track of the users who are allowed to take part in the game.

Additionally, the X-Forge Game Engine provides transport-independence and portability. This means that your application can run across the Internet or a Bluetooth PAN, without any additional coding, and can be ported to other platforms that support the Fathammer libraries.

For more information, see "Chapter 20, Multiplayer Games" in the *X-Forge 1.0.2 Guide*.

7.2. X-Forge Core

The X-Forge Core provides transport-independent networking, using a sockets API. The underlying transport, currently Bluetooth and Internet, is abstracted away.

Four types of packet communications are provided:

- **Guaranteed** - provides automatic retransmission and sequencing
- **Non-guaranteed** - datagram, send-once model
- **Quick-guaranteed** - provides prioritized packets

- **Recent-state** - send at pre-defined intervals

You should use this API if you want to leverage system-provided authentication, and use a socket-oriented machine-to-machine model.

For more information, see "Chapter 5, Overview - Network" in the *X-Forge 1.0.2 Guide*.

7.3. Palm OS

Palm OS contains extensive support for both IP networking and Bluetooth. It provides access to the various protocol layers of the Bluetooth stack, including:

- L2CAP – packet-oriented
- RFCOMM - data-stream/serial-oriented
- OBEX - object-level transfer-oriented

The Palm OS APIs allow for direct control over the stack, including:

- Stack initialization
- Discovering other Bluetooth devices
- Establishing device-to-device links
- Device authentication
- Control of Piconets (8 Bluetooth devices communicating together)

Palm OS also provides access to the IP protocols, TCP and UDP, through a sockets-oriented API. It includes requisite utility functions such as *gethostbyname*.

Medium-level constructs such as Object Exchange and Serial Port Emulation are provided. These can be run over phone/modem, IR, or Bluetooth, to a phone or an Internet Access point.

Note: These routines have no knowledge of gaming objects and are lower-level than those provided by the X-Forge Core API. Utilizing these routines requires a great deal of work, as well as understanding the underlying network protocols. The potential for higher performance must be balanced against the additional development time, code complexity, maintenance, and reduction of portability. If you have a pre-existing implementation of a mechanism to serialize/deserialize gaming objects across a network, you may choose to port your existing code using these routines.

For comprehensive information on Bluetooth, see "Chapter 6, Bluetooth" in the *Palm OS Programmer's Companion, Volume II, Communications*, at:
http://www.palmos.com/dev/support/docs/palmos/Companion2TOC.html .

For full information on TCP/IP networking, as well code samples, see "Chapter 7, Network Communication" in the *Palm OS Programmer's Companion, Volume II, Communications*, at: http://www.palmos.com/dev/support/docs/palmos/Companion2TOC.html.

8. Input

For a thorough description of the Tapwave platform's innovative input event model, see "Advanced Input Support" in the *Tapwave Programmer's Reference*.

For API descriptions, see "Tapwave Input API" in the *Tapwave Programmer's Reference*.

8.1. X-Forge Core

For information on input and controls, see the following sections in the *X-Forge 1.0.2 Guide*:

- "Chapter 3, Platform Independent - Controls"
- "Chapter 5, Overview - Controls"
- "Chapter 6, Application Framework - Controller Input Class"

8.2. Palm OS User Input

The Palm OS user input routines can be used for emulation of a simple, button-oriented game. For this type of application, the navigator can be treated as a four-way or eight-way D Pad (see below for more information on the Tapwave Platform Navigator).

Palm OS user input routines are based on an event queue model. A *GetNextEvent* loop is used to respond to user input events.

Coding with the Palm OS user input routines provide the best portability to simpler Palm OS-based devices, such as those that do not have navigators. Slower input, limited by event loop processing, as well as diminished accuracy, is the price paid for portability in this case.

For a general description of the user interface support in Palm OS, see "Chapter 4, User Interface" in the *Palm OS Programmer's Companion - Volume I*, at: http://www.palmos.com/dev/support/docs/palmos/CompanionTOC.html.

8.3. Tapwave Platform Navigator

Flexible navigator support is built into the Tapwave platform. The navigator can be treated as either a digital or an analog device.

When acting as a digital device, the navigator can function either as a standard Palm™ four-way D pad, or as an eight-way D pad. Generally, with the navigator functioning as a digital device,

Palm OS generates events at a fixed rate. However, the Tapwave platform incorporates multiple navigator thresholds. The further the navigator is pushed towards its maximum value, the faster events are generated. For example, this could be used to move a character from a walk, to a trot, to a run, depending upon the navigator's position along one axis.

For finer control and faster interactivity, you can configure the navigator as an analog device. This is a polling model, in which you check the navigator state as often as required by your application. This provides the fastest possible user feedback.

When used as an analog device, the navigator position returned doesn't take into account the display orientation (landscape versus portrait). The display orientation must be detected and coordinates mapped accordingly. For more information, see "Tapwave Screen APIs" in the *Tapwave Programmer's Reference*.

9. Gaming Support

Tapwave has developed a *High Score API*, which gives you an easy way to manage your user's high scores. This is a special feature of Tapwave devices that enables you to sponsor high score contests, etc., to build community between your game users. A *High Score Conduit* is included, which uploads these scores to Tapwave.com. It is important that you use this High Score API.

For more information, see "Gaming Support" in the *Tapwave Programmer's Reference*, and the *Tapwave High Scores Design* document.

10. Tapwave Platform API Cross-Reference

Tapwave has developed several API sets for Road Dawg to take advantage of its unique features. The table below is a helpful cross-reference, associating desired game features with corresponding sections in the *Tapwave Programmer's Reference.*

Feature	Sections
Screen size and layout	"Large Screen and Landscape Support" "Tapwave Screen API"
Input and event queue management	"Advanced Input Support" "Tapwave Input API"
High Scores	"Gaming Support" "Gaming APIs"
Rumbler	"Vibration Support" "Device API"
Graffiti®/Pen Input Area	"Tapwave Screen API"

Jpeg and Mpeg support	"Music Photo List Manager" document
Photo and Music Playlist support	"Music Photo List Manager" document

11. Development Environment

11.1. Tools

Tapwave provides one of the first Palm OS-certified devices that allow developers to create native-ARM applications without using the Palm OS 5 *Armlets* architecture.

The tools required to develop an ARM application for the Tapwave platform are:

- ARM Development System (ADS), Ver. 1.2, with ADS 842 patch

  - or -

  Metrowerks Codewarrior for ARM, Ver. 1.2, with 1.2.2 upgrade, with the ADS 842 patch (The ADS patch can be downloaded at: http://www.arm.com/support/downloads-ads12.)

  These tools are used to compile applications into native ARM code.

- Microsoft Visual C++, Version 6.0, with Service Pack 5

  Visual C++ is used to build PalmSim version of applications, for testing and debugging.

- CodeWarrior for Palm OS, Version 8.0, upgraded to Version 8.3, then updated with Palm OS SDK 5.1

  CodeWarrior is necessary for both ARM and PalmSim development, providing tools, headers, etc. CodeWarrior can be used to develop a 68K version of applications for testing.

- Tapwave SDK

11.2. SDK Directory Structure

The directory structure of the SDK is:

Incs/ - Tapwave specific header files

Documentation/ - Tapwave developer documentation

PalmSim/ - The PalmSim application and associated Tapwave DLLs.

ROMs/ - Tapwave device ROMs

Samples/ - Tapwave sample code

11.3. Build Instructions

11.3.1. Introduction

Below are instructions for building Tapwave applications in a native ARM environment.

The build process is driven by the JAM development tool. JAM (for "Just Another Make") is very similar to the "make" program, available on many platforms. It automates the building of large projects, often consisting of many source files. JAM uses the files "Jamfile" and "Jamrules" to control the build process.

11.3.2. Build Steps

1. Install *Visual C++ 6.0* with SP5.

   Note: Let the Visual C++ installer setup the environment variables. The Tapwave build environment is not compatible with *Visual C++* version 7. If you use both versions, make sure that the environment variables are setup for *Visual C++ 6.0* only.

2. Install ARM Developer Suite 1.2.
   - or -

   Install Metrowerks' *Codewarrior for ARM* Version 1.2, upgraded to version 1.2.2. The included upgrade file for version 1.2.2 is: `ARM_R1.2.2_Update.exe`.

3. Apply the ADS Compiler Patch 842 to either of the above compilers.
   The included patch file is: `ads12_all_windows_patch.exe.`

4. Install *CodeWarrior for Palm OS 8.0/8.3*, updated with SDK 5.x.
   The included upgrade file is: `Palm OS SDK 5.0.exe.`

5. Set the ADS, VISUALC, and PATH environment variables **using 8.3 pathnames, with no spaces**. One way to set these environment variables is through the SET command lines shown below. Note that the path names in the SET commands specify the default locations of the tools. If you installed the tools in other locations, be sure to set the environment variables to reflect the actual locations.

   Note: The path to *CodeWarrior for Palm OS* must precede the path to *ADS* in the PATH environment variable, otherwise the build will fail. (Type: `PATH` at the command prompt to see your path.)

   If you are using ARM Developer Suite 1.2:

   SET ADS=C:\PROGRA~1\ARM\ADSv1_2

   Or, if you are using *Codewarrior for ARM*:

   SET ADS=C:\PROGRA~1\METROW~1\CODEWA~1

SET VISUALC=C:\PROGRA~1\MICROS~3\VC98

SET PATH=%PATH%;%CD%\Tools

Note: Replace `%CD%` with the top of this build environment directory tree.

Source code for an example application is located in the folder `Tests\Input`.

You should make a backup copy of this example before changing it. It can be built either as a regular Palm 68K application, or as a native ARM application, as described in the next step.

6. Type the following command to build the example application:

   jam -s TARGET={Debug|Release} -s DEVICE={ARM|NT} -s JAM_TOOLSET={ADS|VISUALC}

   Notes:

   - Specify "`TARGET=Debug`" to build a debug version of the application. Specify "`DEVICE=ARM`" to build a native ARM version, or "`DEVICE=NT`" to build a PalmSim version. Specify "`JAM_TOOLSET=ADS`" to build a native ARM version, or "`JAM_TOOLSET=VISUALC`" to build a PalmSim version.
   - The output of this command will be located in the folder "`BuildResults\{ARM|NT}\{Debug|Release}`" For the ARM version the output is a single .prc file. For the NT (PalmSim) version the output is a .prc file and a .dll file.

12. Recommendations

The following are "best practices" recommendations. By following these recommendations, end-users will know what to expect when using your application. Your application will use the Tapwave platform's features effectively, as designed, and will use system resources efficiently.

- Respect the Game Sound Level system preference. See "Get Game Sound Level" in the *Tapwave Programmer's Reference*.
- Only call the High Score Conduit when a new high score has been achieved. The High Score Conduit posts an application's **highest** score to Tapwave.com. This feature gives adroit players public glory and bragging rights. For efficient use of network and battery resources, it is only necessary to call the High Score Conduit when a new, highest score has been achieved.

13. Sample Application

The sample application shows how to quickly port a game to use the Palm APIs and Tapwave APIs. The application demonstrates how to:

- Start the game
- Resize the screen to full landscape mode
- Get navigator input
- Safely draw to the display by using double buffering
- Display an in-game menu when the Home button is pressed

The sample application is distributed as part of the Tapwave SDK. You can use its basic structure as a model for your application. You are welcome to modify it and copy relevant portions into your application.